Contents

1. Overview
2. Design Decisions & Plans
3. Functions

Overview

The frontend of the dashboard is built using Dash by Plotly, a Python framework built on Flask, React, and Plotly.js for building data-related web applications. Dash offers sensible tools and functions to make the development of These web applications are fast and modular, from building app callbacks to designing the site itself. These tools include `html`, which allows us to design and style the app in Python, and Dash's own component library, `dcc`, which offers components such as Graph and Download to display graphs and download data respectively.

The general workflow of the dashboard is akin to a data recorder or monitoring system. There is no incentive to require the dashboard to do any heavy computation or analysis on the data it is receiving and recording. As such, the dashboard's objective is simple: just record the incoming data and display it on a graph. For now, this is done through a WebSocket connection and a toggle switch. When that switch is turned on, the frontend will begin parsing the messages that were sent along the WebSocket connection and display the parsed values on the corresponding graphs on the dashboard. With how the app functions now, the frontend will always receive a message from the WebSocket connection. However, unless the switch is toggled, the message will not be parsed and displayed accordingly.

As of right now, the source code for the frontend of the dashboard exists under the `test/graphs` branch on our GitHub repository.

Design Decisions & Plans

Some of the initial design decisions stem from our sponsor's requirements for this project, and they are subject to change. Namely, we must incorporate their proprietary hardware and software as part of this project. We must use their DuckLinks and their supplementary Duck Management System within our project in some fashion. As we are partnering with UConn's Formula SAE team, OWL Integration's products prove to be useful for our project solution. Their DuckLinks would allow collected data from the racecar, whether from sensors or our machine learning model, to be transmitted directly through the network gateway and toward the DMS. From there, the data would be streamed from the DMS to the dashboard for visualization.

This stream of data, from the car to the dashboard, is our main influence on the design decisions for this project. In our weekly meetings with our sponsor, OWL Integrations, we discuss the possibility of latency in data transmission regarding getting the data from the DMS to the dashboard. Two approaches were discussed: getting data from direct API calls to the DMS or via a WebSocket connection. As of right now, OWL Integrations only has support for direct API calls to the DMS. According to their lead software architect, Charlie Evans, WebSocket support for the DMS will be implemented in the near future. The first approach presents some overhead. The frontend would make a call to the DMS and get some data back after some time. We assume that the time difference between the initial call and the acquisition of data would be unstable and varied enough to warrant a different implementation. Of course, this claim is based on an assumption, but we believe it is a

genuine concern. A WebSocket implementation, on the other hand, would solve this time difference issue, as the data would be received as soon as the DMS receives its data from the DuckLinks. Effectively, the DMS would not act like a middleman the frontend would interact with, but rather a bridge that streams the data from the DuckLinks to the frontend of the dashboard. Since the data from the DuckLinks must be reported to the DMS before being streamed to the frontend, this typecasts the DMS as a data aggregate from which the frontend would pull.

Data visualization relies on two variables, dependent and independent. The dependent variable, being the data coming in from the DMS, can vary within some given range. For example, the car's velocity could range from zero kilometers per hour to the car's top speed. The independent variable will always be time: the time stamp representing when the data was read. However, it is important to choose the interval of the independent variable. If we were to implement the frontend using API calls to the DMS, then it would stand to reason that the frontend should call the DMS periodically, say every second. If we were to implement it with a WebSocket instead, the interval would probably be much less than every second; every 20th of a second, for example. This ultimately depends on the interval of transmission of the DuckLinks, but it is nonetheless important to bring up.

Currently, our dashboard only consists of a frontend. There is no backend or database that the dashboard uses for its operations at the given moment. Initially, we believed that a dashboard frontend would suffice, but this presents numerous problems that would otherwise not be present. With how the code is written in the branch `test/graphs`, the dashboard parses the WebSocket message and appends the data values to its corresponding array in a dictionary. This dictionary is stored in the browser's local storage. The browser's local storage is persistent, meaning that the data would still exist across browser refreshes. This is beneficial to the dashboard, as the data will never be lost. However, this is also detrimental to the dashboard, as the local storage would have to be cleared every time a new recording session starts. The data persists even after a full restart of the dashboard. As such, the local storage solution only exists to compensate for the lack of a backend and database. The values the graph uses and the CSV file that is generated upon clicking the "download" button, depend on the browser's local storage. In addition, the arrays inside the dictionary dynamically increase every second. There is only so much memory that a computer can allocate for the local storage before the browser window inevitably crashes. For demonstration purposes, this is fine, but it will not suffice in actual production. Ideally, the values inside local storage would not change dynamically, but are rather static variables, like authentication keys or cookies. For these reasons, we will implement a backend and database solution for our dashboard.

Functions

This section goes over every function and component used in `main.py` in the `test/graphs` branch of our GitHub repository. Our demo mimics the interaction between the DMS and the dashboard. The DMS would stream the data to the dashboard as a JSON object over a WebSocket connection, to which the frontend would parse that object.

General Imports

```
import json
import pandas as pd
import dash_daq as daq
import plotly.graph_objects as go
from dash_extensions import WebSocket
from dash_extensions.enrich import Dash, Input, Output, dcc, html, State
```

1. JSON
   - A library to handle WebSocket message parsing
2. Pandas
   - A library to convert the dictionary of arrays of integers to a `Pandas.dataFrame`
   - Dash offers support for downloading dataframes as CSV
     - For more information, visit https://dash.plotly.com/dash-core-components/download
3. Dash DAQ
   - A library consisting of data acquisition components made for Dash application development
     - `daq.BooleanSwitch()`
     - For more information, visit https://dash.plotly.com/dash-daq
4. Plotly Graph Objects
   - A library to create, manipulate, and render graphs on the dashboard
     - For more information, visit https://plotly.com/python/graph-objects/
5. Dash Extensions: WebSocket
   - A library consisting of Dash components and python modules; an extension to Dash
   - Includes WebSocket support
     - For more information, visit https://www.dash-extensions.com/components/websocket
6. Dash Extensions: Enrich
   - A library consisting of 'enriched' Dash objects and modules
     - For more information, visit https://www.dash-extensions.com/sections/enrich

Figure initialization

```
def figure(_t: str, _x=[], _y=[]):
    return go.Figure(
            data=go.Scatter(x=_x, y=_x),
            layout=go.Layout(
                title={'text': _t},
                xaxis={'autorange': True, 'rangemode': 'nonnegative'},
                yaxis={'autorange': True, 'rangemode': 'nonnegative'}
            )
        )
```

Synopsis:

- `figure` returns an empty `Figure` graph object, used for initialization.
- `data` is a `Scatter` graph object, with input parameters being `x` and `y` as Python `List` objects respectively.
- `layout` is a `Layout` graph object, defining the layout of the `Figure` object.
  - Defines the title of the graph and the bounds of it; the graph will display non-negative values.

Parameters:

- `_t` (str): the title for the graph object
- `_x` (List): by default, an empty Python array
- `_y` (List): by default, an empty Python array

Returns:

- `Figure` graph object

App initialization

```
app = Dash(__name__)
```

Much like Python Flask, the app must be initialized as a `Dash` object.

App Layout

The layout of the dashboard is under a single HTML `div`. This will cover every HTML object inside this `div`.

```
WebSocket(
        id="ws",
        url="ws://127.0.0.1:5000/data",
        ),
```

- WebSocket initialization.
  - For more information about what WebSocket does, visit https://www.dash-extensions.com/components/websocket

```
html.Div([
        html.H1(
            id="text",
            style={'textAlign': 'center'},
            children=[f"UConn Formula SAE Sample Dashboard"]
            ),
        html.Button("Download as CSV", id="btn_csv"),
        daq.BooleanSwitch(
            id="switch_record",
            on=False,
            label="Record",
            labelPosition="top"
            ),
        html.Button("Clear Data", id="btn_clear"),
    ]),
```

- Basic HTML components.
  - Header titled “UConn Formula SAE Sample Dashboard”
  - Button titled “Download as CSV”
  - Toggle switch titled “Record”, turned off by default
  - Button titled “Clear Data”

```
dcc.Graph(
    id="graph_Voltage",
    figure=figure("Pack Voltage"),
    style={ 'width': '50%', 'height': '50%', 'float': 'left' },
    ),
dcc.Graph(
    id="graph_Current",
    figure=figure("Pack Current"),
    style={ 'width': '50%', 'height': '50%', 'float': 'left' },
    ),
dcc.Graph(
    id="graph_Temperature",
    figure=figure("Pack Temperature"),
    style={ 'width': '50%', 'height': '50%', 'float': 'left' },
    ),
dcc.Graph(
    id="graph_State-of-Charge",
    figure=figure("State of Charge"),
    style={ 'width': '50%', 'height': '50%', 'float': 'left' },
    ),
```

- Four graphs, initialized using `dcc.Graph`.
  - Figures are initialized with the `figure` function defined above
  - For more information about what `dcc.Graph` does, visit https://dash.plotly.com/dash-core-components/graph

```
dcc.Download(id="download-data-csv"),
dcc.Store(
    id='graph_Data',
    storage_type='local',
    data=empty_data()
    ),
dcc.Interval(
    id="clock",
    interval=ONE_SECOND,
    n_intervals=0,
    disabled=True
    )
```

- Defining three components from `dcc`.
  - `dcc.Download`:
    - Allows for user to download files from the dashboard
    - For more information about `dcc.Download`, visit https://dash.plotly.com/dash-core-components/download
  - `dcc.Store`:
    - Allows storage of JSON data in the browser
    - Initialized as a dictionary of lists, in the form of:
      - `{"State of Charge": list(), "Pack Temp": list(), "Pack Voltage": list(), "Pack Current": list()}`
    - For more information about `dcc.Store`, visit https://dash.plotly.com/dash-core-components/store
  - `dcc.Interval`:
    - Allows the dashboard to update periodically without the user's input
    - The interval period is set to one (1) second, or 1000 milliseconds
    - Used in parsing WebSocket messages
    - For more information about `dcc.Store`, visit https://dash.plotly.com/dash-core-components/interval

Callback Functions

Callback functions are "functions that are automatically called by Dash whenever an input component's property changes, in order to update some property in another component (the output)." Since there are lots of moving parts in this dashboard, callback functions make up a majority of the frontend.

```
@app.callback(
        Output("clock", "disabled"),
        Input("switch_record", "on")
        )
def toggle_interval_clock(switch_state):
    return not switch_state
```

Synopsis:

- Toggles the state of the internal clock `Interval` object.
- Whenever the toggle switch is on, meaning that the boolean for `on` is `True`, the `Interval` object's `disabled` property is set to `False`, meaning that internal clock will begin updating periodically.

Parameters:

- `switch_state` (`boolean`): the state of the `BooleanSwitch`'s 'on' property

Returns:

- The opposite of `switch_state`; False → True, True → False

```
@app.callback(
        Output("graph_Data", "data"),
        Input("ws", "message"),
        State("switch_record", "on"),
        State("graph_Data", "data")
        )
def parse_response(e, switch_state, data):
    updated_data = data or empty_data()
    if e == None or switch_state == False:
        return updated_data
    response = json.loads(e['data'])
    updated_data['State of Charge'].append(response['charge'])
    updated_data['Pack Temp'].append(response['temp'])
    updated_data['Pack Voltage'].append(response['volt'])
    updated_data['Pack Current'].append(response['amps'])
    return updated_data
```

Synopsis:

- Reads the WebSocket message and updates the graphs' data accordingly.
- The message is a JSON object in the form below, where `random_sample[i]` is a random number between 0 and 10:

```
{"volt": random_sample[0], "amps": random_sample[1], "temp": random_sample[2],
"charge": random_sample[3]}
```

- 'volt' represents the data for the Pack Voltage graph, 'amps' represents the data for the Pack Current graph, and so on.
- This callback function gets called with every tick update of the internal clock `Interval` object.

Parameters:

- `e` (WebSocket): the WebSocket message
- `switch_state` (`boolean`): the current state of the `BooleanSwitch`'s `on` boolean.
- `data` (dictionary of lists): the current state of the dictionary of lists in the browser's local storage.

Returns:

- `data`, dictionary of lists, with appended values.

```
@app.callback(
        Output("graph_State-of-Charge", "extendData"),
        State("switch_record", "on"),
        State("graph_Data", "data"),
        Input("clock", "n_intervals")
        )
def update_graph_charge(on, data, n_intervals):
    if on==False: return
    latest_charge = data['State of Charge'][-1]
    return (dict(x=[[n_intervals-1]], y=[[latest_charge]]), [0])
```

This callback function represents the other callback functions relating to updating the graph objects. This one in particular updates the "State of Charge" graph, and is indicative of other functions which update "Pack Temp", "Pack Voltage", and "Pack Current" respectively.

Synopsis:

- Reads the most recent (last) value in the "State of Charge" array in `graph_Data` dictionary of lists, and extends the "State of Charge" graph with that value.
- The x-axis represents the number of seconds since the start of the recording.
- The y-axis represents the random value, between 0 and 10, that `producer.py` has generated.
- Since there is only one trace for each respective graph, the values are updated for the first trace, `[0]`.
  - For more information on graphs and how to update them, visit https://dash.plotly.com/dash-core-components/graph

Parameters:

- `n_intervals` (int): seconds since the start of the recording.
- `on` (`boolean`): the current state of the `BooleanSwitch`'s `on` boolean.
- `data` (dictionary of lists): the current state of the dictionary of lists in the browser's local storage.

Returns:

- A tuple containing the dictionary of lists regarding the x and y axis, and the trace that is being updated.

```
@app.callback(
        Output("download-data-csv", "data"),
        Input("btn_csv", "n_clicks"),
        State("graph_Data", "data"),
        prevent_initial_call=True,
        )
def download_data_as_csv(n_clicks, data):
    df = pd.DataFrame(data)
    return dcc.send_data_frame(df.to_csv, f"{date.today()}.csv")
```

Synopsis:

- Converts the dictionary of lists in local storage to a CSV file for download.

Parameters:

- `n_clicks` (int): the number of button clicks.
- `data` (dictionary of lists): the current state of the dictionary of lists in the browser's local storage.

Returns:

- A CSV file

```
@app.callback(
        Output("graph_Data", "data"),
        Output("graph_Voltage", "figure"),
        Output("graph_Current", "figure"),
        Output("graph_Temperature", "figure"),
        Output("graph_State-of-Charge", "figure"),
        Output("clock", "n_intervals"),
        Input("btn_clear", "n_clicks"),
        prevent_initial_call=True,
        )
def clear_local_storage(n_clicks):
    return empty_data(), figure("Pack Voltage"), figure("Pack Current"), figure("Pack
Temperature"), figure("State of Charge"), 0
```

Synopsis:

- Upon clicking of the 'Clear Data' button, resets every value in the dashboard, including the local storage, the graphs, and the internal clock interval.

Parameters:

- `n_clicks` (int): the number of button clicks,

Returns:

- The cleared state of:
  - graph_Data's `data` (dictionary of lists)
  - graph_Voltage's `figure` (`Figure` graph object)
  - graph_Current's `figure` (`Figure` graph object)
  - graph_Temperature's `figure` (`Figure` graph object)
  - graph_State-of-Charge's `figure` (`Figure` graph object)
  - clock's `n_intervals` (int)